

MANUAL26. УРОК 7. ГРАНИЧНЫЕ ЗНАЧЕНИЯ: ПОЧЕМУ ОШИБКИ ЧАЩЕ ЖИВУТ НА «КРАЯХ».

На предыдущем уроке вы научились сокращать количество проверок с помощью эквивалентных классов. Это сильно помогает не утонуть в однотипных сценариях и держать покрытие под контролем. Сегодня делаем следующий шаг. Если эквивалентные классы отвечают на вопрос «какие группы значений вообще есть», то граничные значения отвечают на вопрос «где внутри этих групп вероятность ошибки максимальна».

Что нового в этой теме

Тема урока — граничные значения. Это техника, которая помогает ловить дефекты на стыках диапазонов: около

минимальных и максимальных допустимых значений, а также сразу за их пределами. На практике именно там чаще всего возникают «тихие» ошибки: off-by-one, некорректная валидация, неправильные сообщения об ошибке, сбои в сохранении данных.

Теоретический блок: почему именно «края» чаще всего ломаются

Середина диапазона обычно работает спокойно. Большинство проблем живет не там, а на переходах — в точке, где система переключается с «разрешить» на «запретить». Именно здесь легко ошибиться в одном символе: написать `<` вместо `<=` или наоборот.

Для пользователя такая мелочь превращается в заметный дефект: значение на границе внезапно не проходит, хотя по требованиям должно. Поэтому ошибки часто всплывают именно на `min`, `max` и рядом с ними — `min-1`, `max+1`.

Есть простая инженерная логика: где меняется правило, там выше риск. Граница — это и есть место

смены правила. Вот почему проверка только «средних» значений дает иллюзию надежности, но не защищает релиз от реальных сбоев.

Что такое граничные значения простыми словами

Граничные значения

это числа и параметры около лимитов, которые разделяют "можно" и "нельзя".

Обычно для диапазона нас интересуют:

- минимально допустимое значение;
- значение чуть выше минимума;
- значение чуть ниже минимума;
- максимально допустимое значение;
- значение чуть ниже максимума;
- значение чуть выше максимума.

Ключевая идея проста: мы тестируем не случайные точки, а точки перехода между разными режимами поведения системы.

Такой подход дает высокий шанс быстро найти дефект при небольшом количестве проверок.

Как граничные значения связаны с эквивалентными классами

Эти техники работают в связке. Сначала вы на уроке 6 выделяете классы: валидные и невалидные группы. Потом внутри этих групп находите самые чувствительные точки — границы. Получается не «много тестов ради количества», а структурированный набор с хорошей отдачей.

Можно представить это так:

Эквивалентные классы показывают карту территории, а граничные значения указывают места, где чаще всего случаются аварии.

Когда эти техники используются вместе, QA получает и экономию времени, и более высокую вероятность найти реальные дефекты.

Пример на Apex Legends

Представим, что в Apex Legends добавили турнирный модуль с ограничением по размеру команды: в заявке может быть от 3 до 5 игроков. На первый взгляд правило простое. Но именно здесь границы критичны.

Если тестировать только «типичное» значение 4, можно ошибочно решить, что все работает. Но настоящая проверка начинается на краях:

- 3 игрока (минимум, должно быть принято),
- 2 игрока (ниже минимума, должно быть отклонено),
- 5 игроков (максимум, должно быть принято),
- 6 игроков (выше максимума, должно быть отклонено).

Эти четыре точки уже дают намного больше уверенности, чем любой набор случайных значений в центре диапазона. Если в одном из краевых значений поведение неверное, пользователь столкнется с проблемой сразу, потому что границы часто попадают в реальные сценарии.

**Какие ошибки чаще
всего ловятся на
границах**

1. Первая типовая ошибка — off-by-one, когда система принимает или отклоняет значение на единицу неправильно.
2. Вторая — несогласованность между интерфейсом и бекендом. Например, UI показывает лимит «до 5», а сервер на самом деле режет на 4.
3. Третья — неправильный текст ошибки. Значение отклонено верно, но сообщение вводит пользователя в заблуждение («неверный формат» вместо «превышен лимит»).
4. Четвертая — разные правила в разных местах продукта. В одном экране максимум 16 символов, в другом для того же поля — 15. Такие рассинхронизации особенно хорошо проявляются именно на границах.
5. Пятая — сбои при сохранении и повторном открытии. Значение на границе прошло валидацию при вводе, но после сохранения отобразилось иначе или не восстановилось корректно.

Как применять технику пошагово

Сначала найдите явное правило в требованиях: диапазон, лимит, размер, количество, длина. Если правило не сформулировано точно, зафиксируйте это как риск и уточните, иначе проверка будет строиться на предположениях.

Далее выделите две валидные границы: минимум и максимум. После этого добавьте точки рядом: на шаг ниже минимума, на шаг выше минимума, на шаг ниже максимума, на шаг выше максимума. Для целых чисел это обычно «-1» и «+1» относительно края.

Потом проверьте не только факт «принято/отклонено», но и весь пользовательский сценарий: понятность сообщения, корректность сохранения, отображение после перезахода, отсутствие побочных эффектов.

И финальный шаг — зафиксируйте, какие границы уже покрыты, а какие пока не покрыты в релизном окне. Это важно, чтобы команда не жила в иллюзии полного покрытия.

Где техника особенно полезна

- Граничные значения особенно эффективны для:
- числовых диапазонов (уровень, возраст, рейтинг, ставка, цена);
- длины строк (никнейм, название команды, сообщение в чате);
- количественных лимитов (слоты, участники, предметы в корзине);
- времени и таймаутов (длительность сессии, окно подтверждения);
- размеров файлов и медиа.

В игровом контексте это почти любой параметр с лимитом. И именно поэтому техника так важна для junior QA: она быстро дает практический результат и повышает качество уже на раннем этапе карьеры.

Ограничения техники: что она не заменяет

Граничные значения не заменяют эквивалентные классы, таблицы решений, исследовательское тестирование и здравый смысл. Если проверять только края, можно пропустить логические проблемы внутри сложных комбинаций условий.

Также техника не помогает, если требования неясны. Нельзя проверять «границу», если команда не договорилась, где эта граница проходит.

И еще важно: некоторые дефекты вообще не завязаны на диапазоны, например проблемы ролей, бизнес-правил, синхронизации данных или сетевого поведения. Для них нужны другие подходы.

Типичные ошибки начинающих в этой теме

1. Первая ошибка — проверять только минимум и максимум, забывая точки сразу за пределом. Тогда переход «валидно -> невалидно» остается непроверенным.
2. Вторая — смешивать шаги для разных типов данных. Для целых чисел шаг может быть 1, а для времени или валюты шаг может быть другим. Границы нужно выбирать по реальному формату поля.
3. Третья — проверять только ввод, но не проверять сохранение и повторное отображение. В результате часть дефектов уходит в прод.

4. Четвертая — не фиксировать ожидаемый результат заранее. Без четкого ожидания легко принять ошибочное поведение как «нормальное».
 5. Пятая — считать, что если граничные значения прошли, значит покрыто все. Это сильная, но не универсальная техника.
-

Итоги занятия

На этом уроке мы разобрали, почему «края» диапазонов — самое уязвимое место в проверках и как работать с ними системно. Граничные значения помогают быстро находить дефекты с высокой вероятностью и не тратить время на случайные проверки.

Профессиональный вывод: хороший QA проверяет не только «середину», где обычно все спокойно, а точки перехода, где система меняет поведение. Именно там чаще всего рождаются неприятные дефекты.

Глоссарий к занятию

1. **Граничное значение** – это значение на границе

допустимого диапазона или рядом с ней.

2. **Off-by-one** – это ошибка на единицу в условиях сравнения или ограничениях.
3. **Минимум/максимум** – это нижняя и верхняя границы валидного диапазона.
4. **Валидный диапазон** – это интервал значений, который система должна принимать.
5. **Невалидный диапазон** – это интервал значений, который система должна корректно отклонять.